

Virtual Functions & Polymorphism

Junior / Mid C++ հարցազրույցի խորացված ուղեցույց

Այս մասը ընդգրկում է Virtual Function, Pure Virtual Function, Override vs Overload, vtable, vptr, Dynamic Dispatch, Runtime Polymorphism, Abstract Class, `dynamic_cast` և RTTI թեմաները՝ memory-level բացատրություններով և պատրաստի interview պատասխաններով:

1. Ինչ խնդիր է լուծում Virtual Function-ը

```
class Animal {
public:
    void speak() { cout << "Animal\n"; }
};

class Dog : public Animal {
public:
    void speak() { cout << "Dog\n"; }
};
```

```
Dog d;
d.speak();    // Dog (նորմալ է)

Animal* ptr = new Dog();
ptr->speak();  // Animal (!!) -- ոչ Dog
```

Ինչո՞ւ `Animal`: `ptr` -ի type-ը `Animal*` է, և compiler-ը compile time-ում որոշում է կանչել `Animal::speak()`: Սա կոչվում է **Static Binding / Early Binding**:

Լուծումը — virtual

```
class Animal {
public:
    virtual void speak() { cout << "Animal\n"; }
};

class Dog : public Animal {
public:
    void speak() override { cout << "Dog\n"; }
};

Animal* ptr = new Dog();
ptr->speak();    // Dog
```

Սա **Runtime Polymorphism** է (dynamic binding / late binding):

Polymorphism = Poly (many) + Morph (forms)

Նույն interface (`Speak()`), տարբեր behavior' `Dog -> Dog` , `Cat -> Meow` , `Cow -> Moo` :

2. Override vs Overload

Override — inheritance + virtual

```
class Animal {  
public:  
    virtual void speak() { cout << "Animal"; }  
};  
  
class Dog : public Animal {  
public:  
    void speak() override { cout << "Dog"; }  
};
```

Derived-ը փոխում է Base class-ի behavior-ը՝ նույն signature-ով:

Overriding means redefining a virtual function from the base class in a derived class with the same signature.

override keyword

`void speak() override` — պարտադիր չէ, բայց ցանկալի է: Եթե սխալ գրես (`void speak()` `override`), compiler-ը կբռնի: Առանց `override` -ի սխալ signature-ը կարող էր անցնել unnoticed՝ ստեղծելով նոր function և կոտրելով polymorphism-ը:

Overload — նույն անուն, տարբեր parameters (inheritance պետք չէ)

```
class Test {  
public:  
    void print() { }  
    void print(int x) { }  
    void print(double x) { }  
};
```

Override vs Overload

	Override	Overload
Inheritance	պահանջվում է	պետք չէ
virtual	պահանջվում է	պետք չէ
Signature	նույնը	տարբեր
Resolution	runtime	compile-time

Interview trap

```
class Base {
public:
    virtual void f(int) { }
};

class Derived : public Base {
public:
    void f(double) { }    // ՈՉ override, ՈՉ overload -- պարզապես նոր function
};
```

Signature-ը տարբեր է, ուստի սա override չէ: Ավելին՝ այն «ստվերում» (hide) է անում Base-ի `f`-ը derived scope-ում (name hiding):

3. vtable և vptr

Երբ class-ում կա գոնե մեկ virtual function, compiler-ը ստեղծում է **vtable** (Virtual Table)՝ virtual function-ների հասցեների աղյուսակ:

Animal vtable	Dog vtable
+-----+	+-----+
Animal::speak	Dog::speak
+-----+	+-----+

Յուրաքանչյուր object-ի մեջ compiler-ը գաղտնի ավելացնում է **vptr** (Virtual Pointer), որը ցույց է տալիս class-ի vtable-ին:

```
Animal a;           Dog d;  
[vptr] --> Animal vtable  [vptr] --> Dog vtable
```

Ամենակարևոր տարբերությունը

vtable	->	պատկանում է CLASS-ին (մեկ հատ յուրաքանչյուր polymorphic class-ի համար)
vptr	->	պատկանում է OBJECT-ին (ամեն object-ի ներսում մեկ հատ)

Եթե ունեն `Animal a1, a2, a3;`, vtable-ը չի կրկնապատկվում — կա մեկ `Animal vtable`, բայց երեք vptr (a1, a2, a3-ի ներսում):

Virtual call-ի ժամանակ ինչ է կատարվում

```
Animal* ptr = new Dog();  
ptr->speak();
```

Runtime-ում.

1. վերցրու object-ը
2. կարդա vptr-ը
3. vptr-ով մտիր vtable
4. գտիր speak-ի հասցեն
5. jump արա այդ հասցեին -> Dog::speak()

Առանց virtual-ի

`Animal* ptr = new Dog(); ptr->Speak();` — compiler-ը compile time-ում տեսնում է `Animal*` և ուղղակի կանչում `Animal::Speak()` : vtable/vptr պետք չէ:

Երբ է vptr ստեղծվում

```
class A { public: virtual void f(); }; // ունի vptr
class B { public: void f(); }; // սովորաբար ՉՈՒՆԻ vptr, ոչ vtable
```

4. Memory Overhead & sizeof

Polymorphic class-ի ամեն object-ի մեջ ավելանում է vptr (64-bit համակարգում սովորաբար 8 bytes):

```
class A { int x; }; // ~4 bytes
class B { int x; virtual void f(); }; // ~16 bytes (4 + 8 vptr + alignment)
```

Interview follow-up. `sizeof(B)` -ը մեծ է, քանի որ ամեն object պահում է vptr, և alignment-ը լրացնում է մինչև 16: vtable-ն ինքը object-ի մեջ չէ — object-ում միայն vptr-ն է:

5. Pure Virtual Function & Abstract Class

```
class Animal {
public:
    virtual void speak() = 0;    // = 0 -> pure virtual
};
```

`= 0` նշանակում է 'Base class-ը implementation չունի, և derived class-ը պարտավոր է override անել: Class-ը դառնում է **Abstract**.

```
Animal a;    // ERROR: cannot instantiate abstract class

class Dog : public Animal {
public:
    void speak() override { cout << "Dog"; }
};

Dog d;    // OK
```

Իրական օրինակ

```
class Shape {
public:
    virtual double area() = 0;
};

class Circle : public Shape {
public:
    double area() override { return 3.14 * r * r; }
};

// Shape s;    -> ERROR
// Circle c;    -> OK
```

Pure Virtual Destructor

Հետաքրքիր հարց՝ pure virtual destructor-ը պարտավոր է ունենալ implementation.

```

class Base {
public:
    virtual ~Base() = 0;
};

Base::~Base() { cout << "Base dtor\n"; } // implementation ՊԱՐՏԱԴԻՐ է

```

Ինչո՞ւ implementation է պետք: Երբ derived object-ը ոչնչանում է, հերթականությունը `Derived::~~Derived()` -> `Base::~~Base()` է, ուստի Base-ի destructor-ը պետք է գոյություն ունենա: Օգտագործում ենք, երբ ուզում ենք class-ը լինի abstract, բայց միաժամանակ ունենա virtual destructor.

```

class Shape {
public:
    virtual ~Shape() = 0;
};

Shape::~Shape() {}

// Shape s; // -> ERROR (abstract)
// class Circle : public Shape {}; Circle c; // -> OK

```

6. dynamic_cast & RTTI

`dynamic_cast` -ը RTTI-ի (Run Time Type Information) թեմայից է. runtime-ում ստուգում է՝ base pointer-ի տակ իսկապես որ derived type-ն է:

```
class Animal {
public:
    virtual ~Animal() {}    // polymorphic լինելու համար պետք է virtual function
};

class Dog : public Animal {
public:
    void bark() { cout << "Bark"; }
};
```

Success

```
Animal* p = new Dog();
Dog* d = dynamic_cast<Dog*>(p);    // p-ի տակ Dog կա -> վերադարձնում է valid Dog*
if (d) d->bark();                  // OK
```

Failure

```
class Cat : public Animal {};
```



```
Animal* p = new Cat();
Dog* d = dynamic_cast<Dog*>(p);    // p-ի տակ Cat է -> nullptr
if (d) d->bark();                  // չի կատարվում
```

Ինչու ոչ static_cast

```
Animal* p = new Cat();
Dog* d = static_cast<Dog*>(p);    // compile-ը անցնում է, բայց runtime-ը UNDEFINED
```

`static_cast` -ը runtime check չի անում. սխալ type-ի դեպքում undefined behavior:

`dynamic_cast` -ը անվտանգ է՝ failure-ի դեպքում `nullptr` (pointer-ի), կամ `std::bad_cast` (reference-ի) է տալիս:

Ինչու է virtual function պետք dynamic_cast-ին

```
class Animal { };           // ոչ polymorphic
Animal* p = new Dog();
dynamic_cast<Dog*>(p);       // COMPILER ERROR (RTTI չկա)
```

`dynamic_cast` -ը RTTI-ի կարիք ունի, իսկ RTTI-ն կա միայն polymorphic class-ների համար (զոնե մեկ virtual function): Ավելացրու `virtual ~Animal() {}` և `dynamic_cast`-ը կաշխատի:

Ինչպես է աշխատում

Object-ում կա `[vptr][data]`; `vptr`-ը ցույց է տալիս `vtable`-ին, որտեղ `compiler`-ը պահում է նաև RTTI տվյալները (type info): `dynamic_cast` -ը՝ `vptr` -> RTTI -> ստուգում իրական type-ը -> որոշում `cast`-ը legal է թե ոչ:

7. Լրացուցիչ Interview թակարդներ

final keyword

```
class Base {
public:
    virtual void f();
};

class Derived : public Base {
public:
    void f() final;    // ավելի ցածր class-ը այլևս չի կարող override անել
};

class Base2 final { };    // Base2-ից inherit անել չի կարելի
```

Virtual call constructor/destructor-ի ներսում

```
class Base {
public:
    Base() { init(); }                // ATTENTION
    virtual void init() { cout << "Base init"; }
};

class Derived : public Base {
public:
    void init() override { cout << "Derived init"; }
};

Derived d;    // Output: "Base init" (ՈՉ Derived init)
```

Constructor/destructor-ի ներսում virtual dispatch-ը «կիմանա» միայն ընթացիկ class-ի մասին, քանի որ derived մասը դեռ չի կառուցվել (կամ արդեն քանդվել է): Դա famous trap է. **երբեք մի հույս դիր virtual call-ի վրա constructor/destructor-ի ներսում:**

typeid (RTTI-ի մյուս կողմը)

```
Animal* p = new Dog();  
cout << typeid(*p).name();    // "Dog" (polymorphic class-ի համար)
```

`typeid` -ն էլ RTTI է օգտագործում; polymorphic type-ի դեպքում `typeid(*p)` տալիս է իրական (Dog) type-ը, ոչ static (Animal)-ը:

Ամփոփ Interview Answers

What is a virtual function? — A virtual function enables runtime polymorphism; the call is resolved at runtime based on the actual object type.

What is a pure virtual function? — Declared with `= 0`; makes the class abstract; derived classes must implement it.

What is overriding? — Providing a new implementation for a virtual function in a derived class with the same signature.

What is overloading? — Multiple functions with the same name but different parameter lists (compile-time).

What is vtable? — A compiler-generated per-class table containing addresses of virtual functions.

What is vptr? — A hidden per-object pointer that points to the class's vtable.

When is vptr created? — When a class has at least one virtual function.

Why virtual destructor? — To ensure the correct derived destructor runs when deleting through a base pointer.

What is dynamic_cast? — A runtime-checked cast; returns a valid pointer on success, `nullptr` on failure (or throws `bad_cast` for references).

What is RTTI? — Run Time Type Information; used by `dynamic_cast` and `typeid` to inspect an object's real type at runtime.

Cheat Sheet

```
vtable      -> class level, one per polymorphic class
vptr        -> object level, points to vtable
virtual call -> object -> vptr -> vtable -> Derived::f()
= 0         -> pure virtual -> abstract class
pure virtual dtor -> still needs a body
dynamic_cast -> runtime check; success=ptr, fail=nullptr
RTTI        -> needs at least one virtual function
final       -> block further overriding / inheritance
ctor/dtor   -> virtual dispatch resolves to current class only
```

Հաջորդ բնական քայլերն են՝ `Rule of Three/Five`, `unique_ptr`, `shared_ptr`, `weak_ptr` և exact memory layout-ի խորացումը: